

[例3.1] 一元多项式及其运算。

一元多项式：
$$f(x) = a_0 + a_1x + \cdots + a_{n-1}x^{n-1} + a_nx^n$$

主要运算：多项式相加、相减、相乘等

【分析】多项式的关键数据是：多项式项数n、每一项的系数a_i（及相应指数i）。有3种不同的方法。

方法1：采用顺序存储结构直接表示

例如：
$$f(x) = 4x^5 - 3x^2 + 1$$

表示成：

下标i	0	1	2	3	4	5	
a[i]	1	0	-3	0	0	4	

方法2：采用顺序存储结构表示多项式的非零项。

- ❖ 相加过程：
- 每个非零项 $a_i x^i$ 涉及两个信息：指数 i 和系数 a_i 。
 - 可以将一个多项式看成是一个 (a_i, i) 元组的集合。
 - 比较 $(9, 12)$ 和 $(26, 19)$ ，将 $(26, 19)$ 移到结果多项式；
 - 继续比较 $(9, 12)$ 和 $(-4, 8)$ ，将 $(9, 12)$ 移到结果多项式；
 - 比较 $(15, 8)$ 和 $(-4, 8)$ ， $15 + (-4) = 11$ ，不为0，将新的一项 $(11, 8)$ 增加到结果多项式；
 - 例如：比较 $(3, 2)$ 和 $(-13, 6)$ ，将 $(-13, 6)$ 移到结果多项式；
 - 比较 $(3, 2)$ 和 $(82, 0)$ ，将 $(3, 2)$ 移到结果多项式；
 - 将 $(82, 0)$ 直接移到结果多项式。
 - 最后得到的结果多项式是： $((26, 19), (9, 12), (11, 8), (-13, 6), (3, 2), (82, 0))$
 - 表示成：
$$P_2(x) = 26x^{19} + 9x^{12} + 11x^8 - 13x^6 + 3x^2 + 82$$

数组下标i	0	1	2	
系数	9	15	3	—
指数	12	8	2	—

(a) $P_1(x)$

数组下标i	0	1	2	3	
系数	26	-4	-13	82	—
指数	19	8	6	0	—

(b) $P_2(x)$

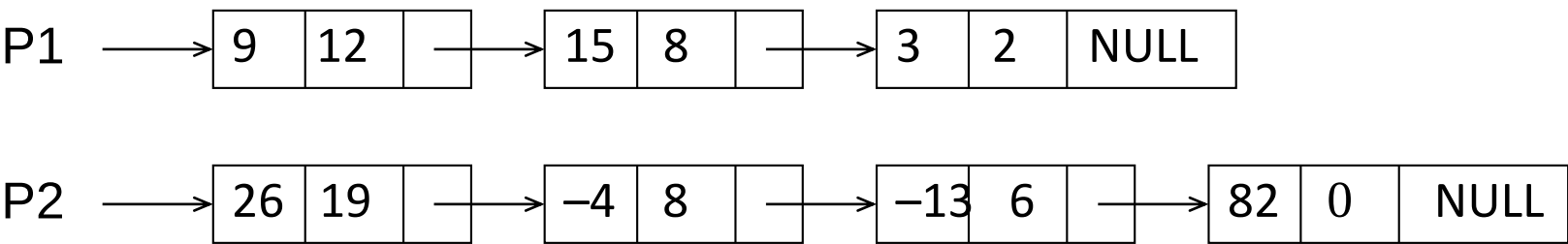
方法3：采用链表结构来存储多项式的非零项。

每个链表结点存储多项式中的一个非零项，包括系数和指数两个数据域以及一个指针域，表示为：

coef	expon	link
------	-------	------

```
typedef struct PolyNode *Polynomial;
typedef struct PolyNode {
    int coef;
    int expon;
    Polynomial link;
}
```

例如：
 $P_1(x) = 9x^{12} + 15x^8 + 3x^2$
 $P_2(x) = 26x^{19} - 4x^8 - 13x^6 + 82$
 链表存储形式为：



[例3.2] 二元多项式又该如何表示？

比如，给定二元多项式： $P(x,y)=9x^{12}y^2+4x^{12}+15x^8y^3-x^8y+3x^2$

【分析】 可以将上述二元多项式看成关于x的一元多项式

$$P(x,y)=(9y^2+4)x^{12}+(15y^3-y)x^8+3x^2$$

所以，上述二元多项式可以用“复杂”链表表示为：

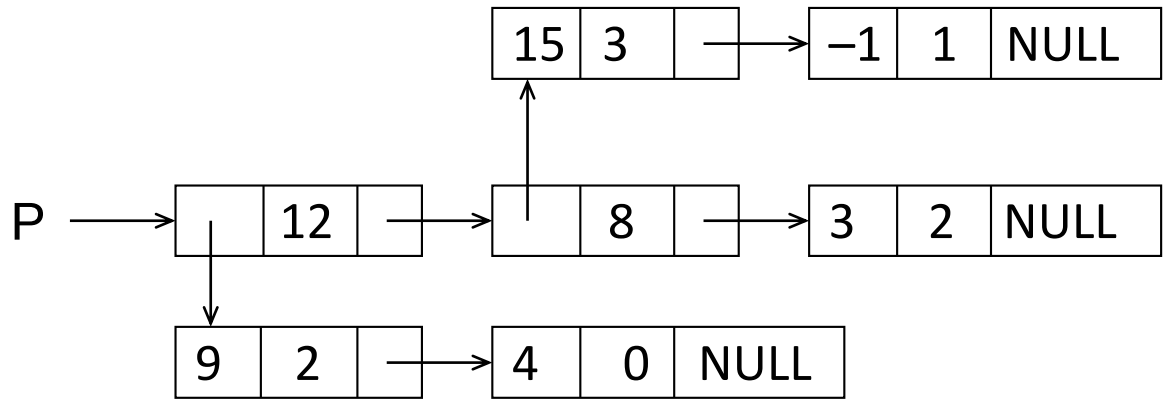


图3.4 二元多项式非零项的链表表示

【定义】“**线性表(Linear List)**”是由同一类型的数据元素构成的**有序序列**的线性结构。

- 线性表中元素的个数称为线性表的**长度**；
- 当一个线性表中没有元素（长度为0）时，称为**空表**；
- 表的起始位置称**表头**，表的结束位置称**表尾**。

类型名称：线性表（List）

数据对象集：线性表是 n (≥ 0) 个元素构成的有序序列 $(a_1, a_2, \dots, a_n)$ ； a_{i+1} 称为 a_i 的直接后继， a_{i-1} 为 a_i 的直接前驱；直接前驱和直接后继反映了元素之间一对一的邻接逻辑关系。

操作集：对于一个具体的线性表 $L \in \text{List}$ ，一个表示位置的整数 i ，一个元素 $X \in \text{ElementType}$ ，线性表的基本操作主要有：

- 1、**List MakeEmpty()**：初始化一个新的空线性表 L ；
- 2、**ElementType FindKth(int K, List L)**：根据指定的位序 K ，返回相应元素；
- 3、**int Find(ElementType X, List L)**：已知 X ，返回线性表 L 中与 X 相同的第一个元素的相应位序 i ；若不存在则返回空；
- 4、**void Insert(ElementType X, int i, List L)**：指定位序 i 前插入一个新元素 X ；
- 5、**void Delete(int i, List L)**：删除指定位序 i 的元素；
- 6、**int Length(List L)**：返回线性表 L 的长度 n 。

❖ 线性表的顺序存储实现

在内存中用地址连续的一块存储空间顺序存放线性表的各元素。一维数组在内存中占用的存储空间就是一组连续的存储区域。

下标 i	0	1	...	$i-1$	i	...	$n-1$	...	MAXSIZE-1
Data	a_1	a_2	...	a_i	a_{i+1}	...	a_n	...	-

↙ Last

```
typedef struct {
    ElementType Data[MAXSIZE];
    int Last;
```

```
} List;
```

```
List L, *PtrL;
```

访问下标为 i 的元素: $L.Data[i]$ 或 $PtrL->Data[i]$
线性表的长度: $L.Last+1$ 或 $PtrL->Last+1$

❖ 主要操作的实现

1. 初始化（建立空的顺序表）

```

List *MakeEmpty( )
{
    List *PtrL;
    PtrL = (List *)malloc( sizeof(List) );
    PtrL->Last = -1;
    return PtrL;
}
    
```

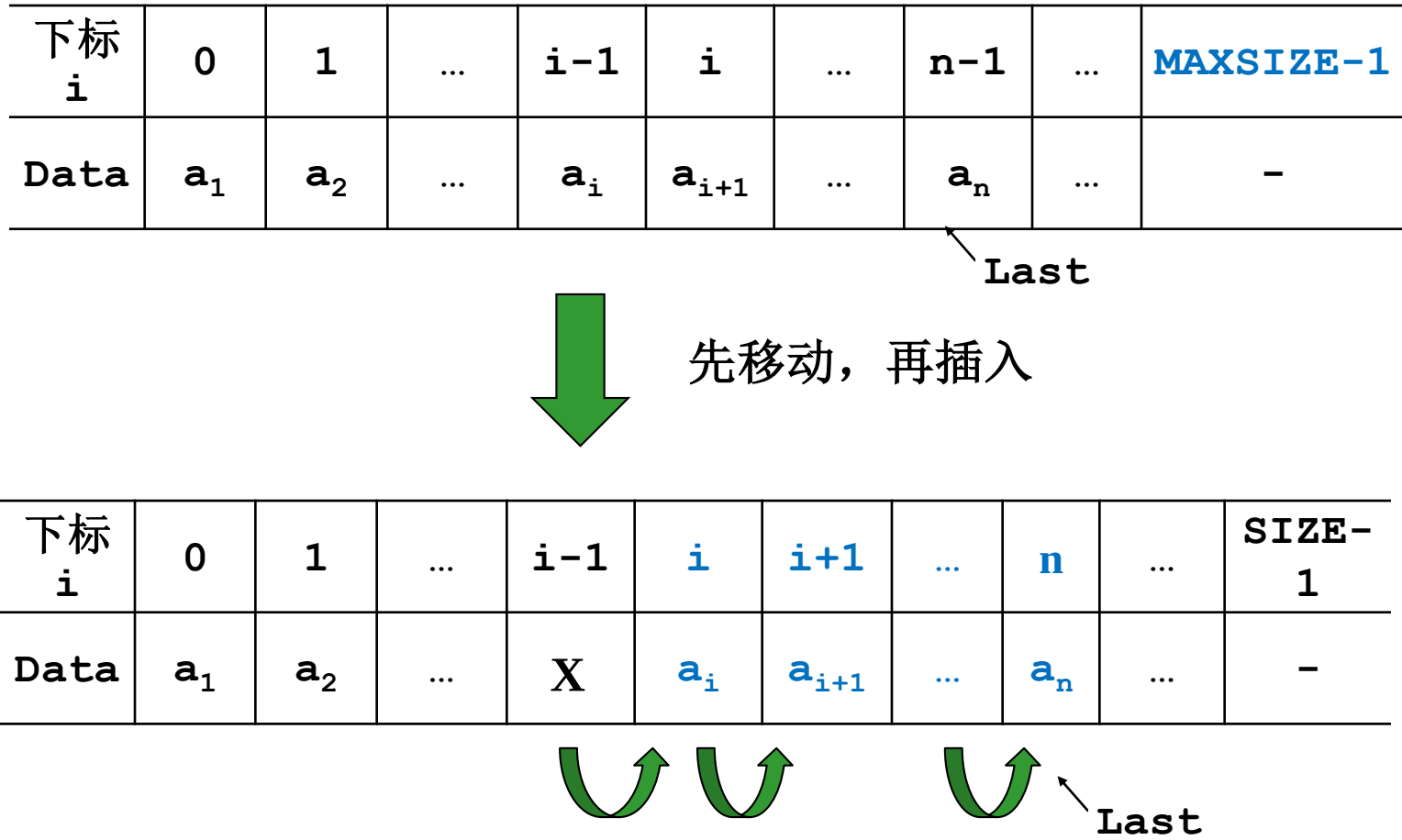
查找成功的平均比较次数为
 $(n+1)/2$ ，平均时间性能为
 $O(n)$ 。

2. 查找

```

int Find( ElementType X, List *PtrL )
{
    int i = 0;
    while( i <= PtrL->Last && PtrL->Data[i] != X )
        i++;
    if ( i > PtrL->Last)    return -1; /* 如果没找到，返回-1 */
    else return i; /* 找到后返回的是存储位置 */
}
    
```

3. 插入 (第 i ($1 \leq i \leq n+1$) 个位置上插入一个值为X的新元素)

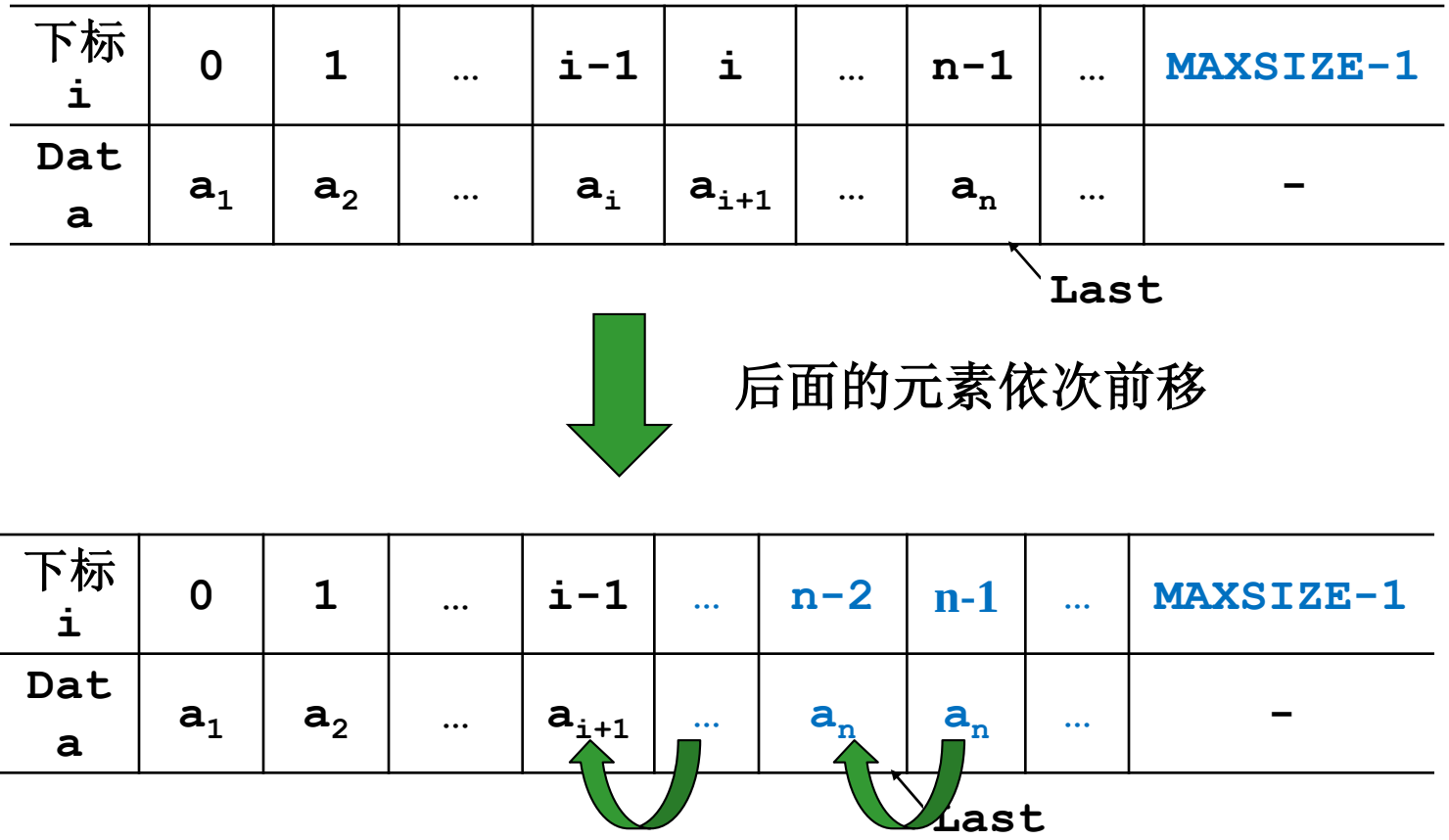


❖ 插入算法

```
void Insert( ElementType X, int i, List *PtrL )
{
    int j;
    if ( PtrL->Last == MAXSIZE-1 ) { /* 表空间已满，不能插入 */
        printf( " 表满 " );
        return;
    }
    if ( i < 1 || i > PtrL->Last+2 ) { /* 检查插入位置是否合法 */
        printf( " 位置不合法 " );
        return;
    }
    for ( j = PtrL->Last; j >= i-1; j-- )
        PtrL->Data[j+1] = PtrL->Data[j]; /* 将  $a_i \sim a_n$  倒序向后移动 */
    PtrL->Data[i-1] = X; /* 新元素插入 */
    PtrL->Last++; /* Last 仍指向最后元素 */
    return;
}
```

平均移动次数为 $n/2$ ，平均
时间性能为 $O(n)$ 。

4. 删除 (删除表的第 i ($1 \leq i \leq n$) 个位置上的元素)



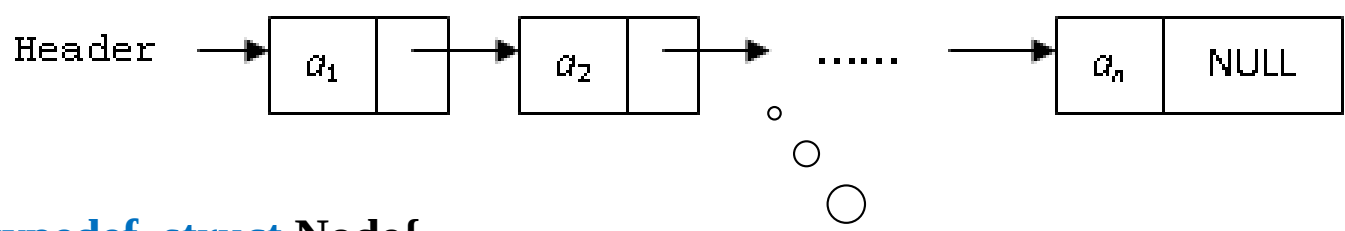
❖ 删除算法

```
void Delete( int i, List *PtrL )
{
    int j;
    if( i < 1 || i > PtrL->Last+1 ) {
        printf( “不存在第%d个元素” , i );
        return ;
    }
    for ( j = i; j <= PtrL->Last; j++ )
        PtrL->Data[j-1] = PtrL->Data[j]; /*将  $a_{i+1} \sim a_n$  顺序向前移动*/
    PtrL->Last--; /*Last仍指向最后元素*/
    return;
}
```

平均移动次数为 $(n-1)/2$ ，
平均时间性能为 $O(n)$ 。

❖ 线性表的链式存储实现

不要求逻辑上相邻的两个数据元素物理上也相邻，它是通过“链”建立起数据元素之间的逻辑关系。因此对线性表的插入、删除不需要移动数据元素，只需要修改“链”。



```

typedef struct Node{
    ElementType Data;
    struct Node *Next;
} List;
List L, *PtrL;
    
```

访问序号为 i 的元素?
求线性表的长度?

❖ 主要操作的实现

1. 求表长

```
int Length ( List *PtrL )  
{ List *p = PtrL;    /* p指向表的第一个结点*/  
  int j = 0;  
  while ( p ) {  
    p = p->Next;  
    j++;              /* 当前p指向的是第 j 个结点*/  
  }  
  return j;  
}
```

时间性能为 $O(n)$ 。

2. 查找

(1) 按序号查找: FindKth;

(2) 按值查找: Find

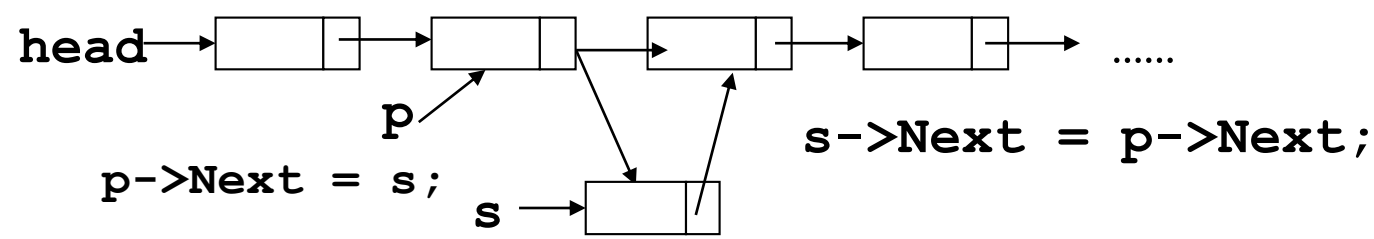
```
List *FindKth( int K, List *PtrL )
{
    List *p = PtrL;
    int i = 1;
    while (p != NULL && i < K) {
        p = p->Next;
        i++;
    }
    if ( i == K ) return p;
    /* 找到第K个, 返回指针 */
    else return NULL;
    /* 否则返回空 */
}
```

```
List *Find( ElementType X, List *PtrL )
{
    List *p = PtrL;
    while ( p != NULL && p->Data != X )
        p = p->Next;
    return p;
}
```

平均时间性能为 $O(n)$ 。

3. 插入 (在链表的第 $i-1$ ($1 \leq i \leq n+1$) 个结点后插入一个值为X的新结点)

- (1) 先构造一个新结点, 用s指向;
- (2) 再找到链表的第 $i-1$ 个结点, 用p指向;
- (3) 然后修改指针, 插入结点 (p之后插入新结点是 s)



思考: 修改指针的两个步骤如果交换一下, 将会发生什么?

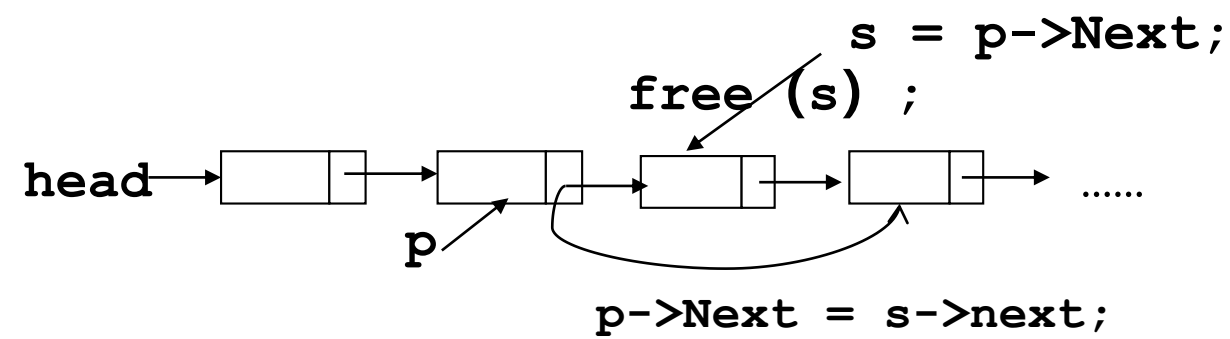
```
List *Insert( ElementType X, int i, List *PtrL )
{
    List *p, *s;
    if ( i == 1 ) { /* 新结点插入在表头 */
        s = (List *)malloc(sizeof(List)); /* 申请、填充结点 */
        s->Data = X;
        s->Next = PtrL;
        return s; /* 返回新表头 */
    }
    p = FindKth( i-1, PtrL ); /* 查找第i-1个结点 */
    if ( p == NULL ) { /* 第i-1个不存在，不能插入 */
        printf( " 参数i错误\n" );
        return NULL;
    }
    else {
        s = (List *)malloc(sizeof(List));
        s->Data = X;
        s->Next = p->Next; /* 新结点插入在第i-1个结点的后面 */
        p->Next = s;
        return PtrL;
    }
}
```

空表时的插入要作为特例，除非单链表带虚头结点。

平均查找次数为 $n/2$ ，平均时间性能为 $O(n)$ 。

4. 删除 (删除链表的第 i ($1 \leq i \leq n$) 个位置上的结点)

- (1) 先找到链表的第 $i-1$ 个结点，用 p 指向；
- (2) 再用指针 s 指向要被删除的结点 (p 的下一个结点)；
- (3) 然后修改指针，删除 s 所指结点；
- (4) 最后释放 s 所指结点的空间。



思考：操作指针的几个步骤如果随意改变，将会发生什么？

❖ 删除算法

```
List *Delete( int i, List *PtrL )
{
    List *p, *s;
    if ( i == 1 ) {
        s = PtrL;
        PtrL = PtrL->Next;
        free(s);
        return PtrL;
    }
    p = FindKth( i-1, PtrL );
    if ( p == NULL ) {
        printf("第%d个结点不存在", i-1);
        return NULL;
    } else if ( p->Next == NULL ) {
        printf("第%d个结点不存在", i-1);
        return NULL;
    } else {
        s = p->Next;
        p->Next = s->Next;
        free(s);
        return PtrL;
    }
}
```

第一个结点的删除要作为特例，除非单链表带虚头结点。

平均查找次数为 $n/2$ ，平均时间性能为 $O(n)$ 。

❖ 广义表

【例】如何表示一个单位的人员情况。一种简单的表示方法是用一个线性表来表示，其先后顺序按照进单位的时间顺序排列：

(张三, 李四, 王五, 钱六, 孙七,)

➤ 如何体现三个不同部门？比如办公室、生产部、销售部。同一个部门放在一起。那么可以用三个有序序列的子表构成的线性表来表示：

((张三,), (李四, 孙七,), (王五, 钱六,))

➤ 如果想表示这个单位的负责人是谁，可将负责人作为表的第一元素：

(丁一, (张三,), (李四, 孙七,), (王五, 钱六,))

【定义】上述这类表就是一种“广义表(Generalized List)”。

➤ 广义表是线性表的推广。

➤ 广义表与线性表一样，也是由n个元素组成的有序序列。

➤ 不同点在于，对于线性表而言，n个元素都是基本的单元素。

➤ 而在广义表中，这些元素不仅可以是单元素也可以是另一个广义表。

❖ 广义表的数据结构可以定义如下：

```
typedef struct GNode{
    int Tag; /*标志域： 0表示该结点是单元素， 1表示该结点是广义表 */
    union { /* 子表指针域Sublist与单元素数据域Data复用， 即共用存储空间 */
        ElementType  Data;
        struct GNode *SubList;
    } URegion;
    struct GNode *Next; /* 指向后继结点 */
} GList;
```

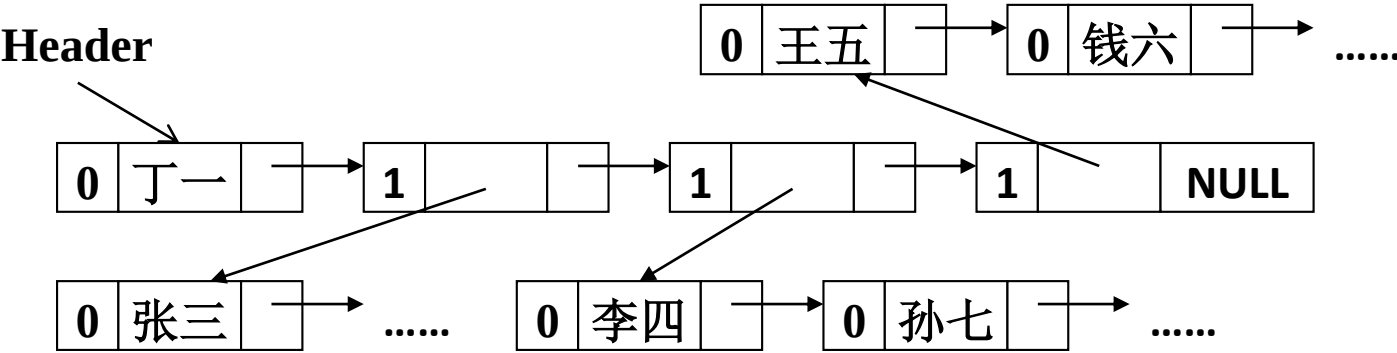
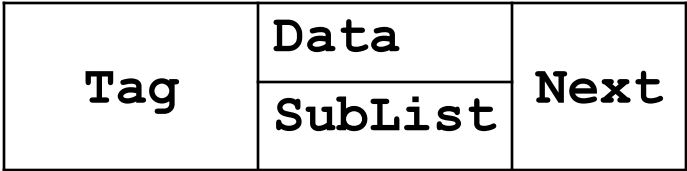


图3.1 广义表结构

❖ 多重链表

【定义】在图3.1的例子中，广义表采用链表存储的方式实现。像这种链表，其元素可能还是另一个子链表的起点指针，叫“多重链表”。

- 一般来说，多重链表中每个结点的指针域会有多个，如前面的例子包含了Next和SubList两个指针域。
- 但包含两个指针域的链表并不一定是多重链表，比如双向链表不是多重链表。
- 多重链表在数据结构实现中有广泛的用途，基本上如树、图这样相对复杂的数据结构都可以采用多重链表的方式实现存储。

[例3.3] 矩阵可以用二维数组表示，但二维数组表示有两个缺陷：

- 一是数组的**大小需要事先确定**，
 - 另一个是当矩阵包含许多0元素时，将造成大量的**存储空间浪费**。
- 例如，对于下面A和B这样的“**稀疏矩阵**”最好是**只存储非0元素**。
如何用多重链表方式实现存储？

$$A = \begin{bmatrix} 18 & 0 & 0 & 2 & 0 \\ 0 & 27 & 0 & 0 & 0 \\ 0 & 0 & 0 & -4 & 0 \\ 23 & -1 & 0 & 0 & 12 \end{bmatrix} \quad B = \begin{bmatrix} 0 & 2 & 11 & 0 & 0 & 0 \\ 3 & -4 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 9 & 13 & 0 \\ 0 & -2 & 0 & 0 & 10 & 7 \\ 6 & 0 & 0 & 5 & 0 & 0 \end{bmatrix}$$

【分析】 采用一种典型的多重链表——**十字链表**来存储稀疏矩阵。

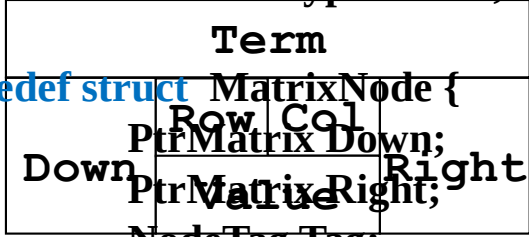
- 链表中用于存放矩阵非0元素的每个结点有**两个指针域**：
一个是行指针(或称为向右指针)**Right**，
另一个是列指针（或称为向下指针）**Down**。
- 结点的**数据域**存放元素的行坐标Row、列坐标Col和数值Value。

➤ 用一个标识域Tag来区分头结点和非0元素结点：
头节点的标识值为“Head”，矩阵非0元素结点的标识值为“Term”。

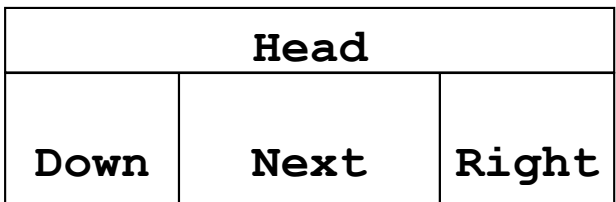
➤ 稀疏矩阵的数据结构可定义为：

```
#define MAXSIZE 50 /* 矩阵最大非0元素个数 */
typedef enum { Head, Term } NodeTag;
typedef struct MatrixNode *PtrMatrix;
typedef struct TermNode {
    int Row;
    int Col;
    ElementType Value;
} TermNode;
typedef struct MatrixNode {
    int Row, Col;
    PtrMatrix Down;
    PtrMatrix Right;
    NodeTag Tag;
} MatrixNode;
union {
    TermNode Term;
    MatrixNode *PtrMatrix;
} URegion;
PtrMatrix HeadNode [MAXSIZE];
```

(a) 结点的总体结构



(b) 矩阵非0元素结点



(c) 头结点

- 用一个标识域Tag来区分头结点和非0元素结点：
- 头节点的标识值为“Head”，矩阵非0元素结点的标识值为“Term”。
- 稀疏矩阵的数据结构可定义为：

```
#define MAXSIZE 50    /* 矩阵最大非0元素个数 */
typedef enum { Head, Term } NodeTag;
typedef struct MatrixNode *PtrMatrix;
typedef struct TermNode {
    int Row;
    int Col;
    ElementType Value;
};
typedef struct MatrixNode {
    PtrMatrix Down;
    PtrMatrix Right;
    NodeTag Tag;
    union {
        PtrMatrix Next;
        TermNode Term;
    } URegion;
};
PtrMatrix HeadNode [MAXSIZE];
```

❖ 矩阵A的多重链表图

